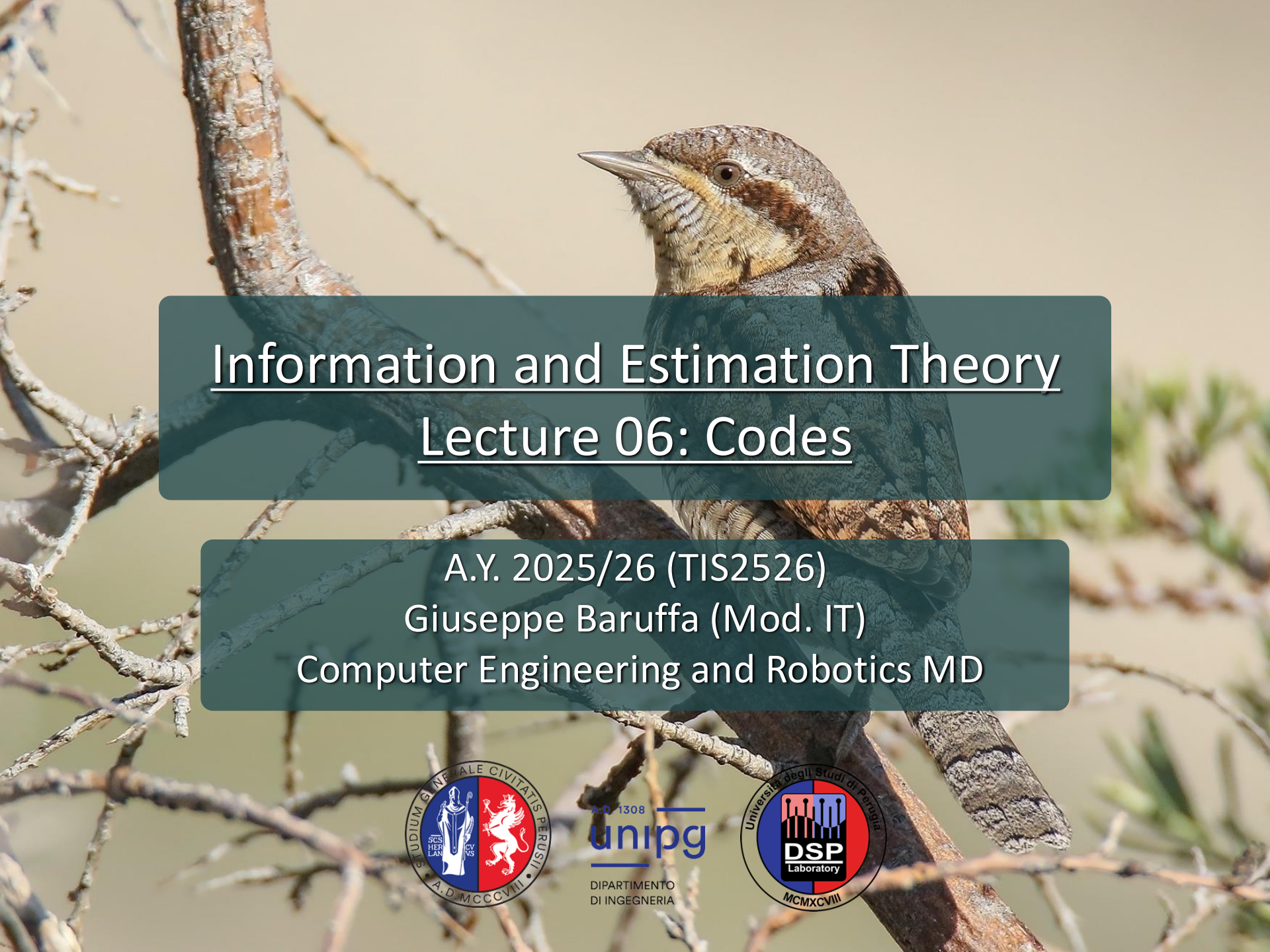


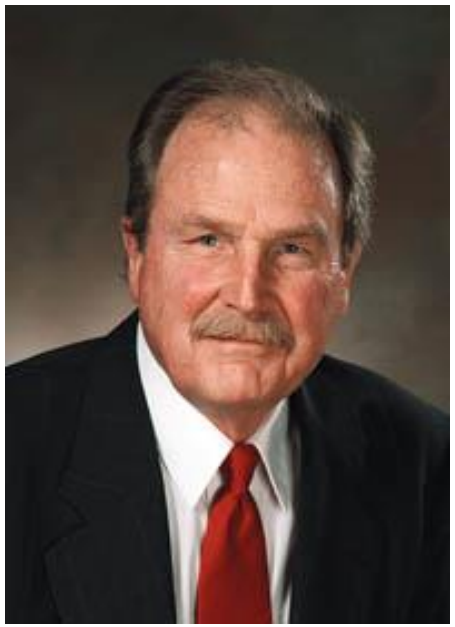
Information and Estimation Theory
Lecture 06: Codes

A.Y. 2025/26 (TIS2526)
Giuseppe Baruffa (Mod. IT)
Computer Engineering and Robotics MD

STUDIUM GENERALE CIVITATIS PERUSII
A.D. MCCCVIII
A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA
Università degli Studi di Perugia
DSP
Laboratory
MCMXCVIII

Huffman Codes

- An optimal (shortest expected length) prefix code for a given PMF can be constructed by a simple algorithm discovered by David Huffman in the ‘50s
- We will prove that any other code for the same alphabet cannot have a lower expected length than this code
- Before we give any formal proofs, let us introduce Huffman codes with some examples



David Huffman (1925–1999)
1999 IEEE Richard W. Hamming Medal

Huffman Code Example

- Take a RV X with $\mathcal{X} = \{1,2,3,4,5\}$ and PMF $p(x)$
- The optimal binary code has the longest CWs for symbols 4 and 5
- These two lengths must be equal, otherwise we could delete a bit from the longer one and have a prefix code with a shorter length
  - We can get a code with the two longest CWs differing only in the last bit
- Combine the symbols 4 and 5 together, to obtain a super-symbol with total probability 0.3
- Proceed by combining the two least likely symbols into one symbol until we are left with one symbol

x	$p(x)$	$l^*(x) = -\log p(x)$
1	0.25	2
2	0.25	2
3	0.2	2.32
4	0.15	2.74
5	0.15	2.74

x	$p(x)$
1	0.25
2	0.25
3	0.2
{4,5}	0.3

Huffman Code Example

x	$p(x)$
1	0.25
{2,3}	0.45
{4,5}	0.3

x	$p(x)$
{{4,5},1}	0.55
{2,3}	0.45

x	$p(x)$
{{{4,5},1},{2,3}}	1

- Build CWs by looping back through each combine, assign 0 and 1 to each combination part

x	$c(x)$
{{{4,5},1},{2,3}}	—

x	$c(x)$
{{4,5},1}	0
{2,3}	1

x	$c(x)$
{4,5}	00
1	01
2	10
3	11

x	$c(x)$
4	000
5	001
1	01
2	10
3	11

x	$c(x)$	$l(x)$	$p(x)$	$l^*(x)$
1	01	2	0.25	2
2	10	2	0.25	2
3	11	2	0.2	2.32
4	000	3	0.15	2.74
5	001	3	0.15	2.74

- This code has average length

$$L = 2 \cdot 2 \cdot 0.25 + 2 \cdot 0.2 + 2 \cdot 3 \cdot 0.15 = 2.3 \text{ bits}$$

- Source entropy is

$$H(X) = 2 \cdot 2 \cdot 0.25 + 2.32 \cdot 0.2 + 2 \cdot 2.74 \cdot 0.15 = 2.286 \text{ bits}$$

Ternary Code Example

- Consider a ternary code (*D*=3) for the same RV
- Combine the 3 least likely symbols into one super-symbol

x	$p(x)$	$l^*(x) = -\log_3 p(x)$
1	0.25	1.26
2	0.25	1.26
3	0.2	1.47
4	0.15	1.73
5	0.15	1.73

x	$p(x)$
1	0.25
2	0.25
{3,4,5}	0.5

x	$p(x)$
{{3,4,5},1,2}	1

x	$c(x)$
{{3,4,5},1,2}	—

x	$c(x)$
{3,4,5}	0
1	1
2	2

x	$c(x)$
3	00
4	01
5	02
1	1
2	2

x	$c(x)$	$l(x)$	$p(x)$	$l^*(x)$
1	1	1	0.25	1.26
2	2	1	0.25	1.26
3	00	2	0.2	1.47
4	01	2	0.15	1.73
5	02	2	0.15	1.73

- This code has an average length *L*=1.5 ternary digits (*trits*) or *L*=2.38 equivalent bits
  - 1 trit = log 3 = 1.585 bits

Other Ternary Example

- If $D \geq 3$, we may not have enough symbols to combine them D at a time
- In this case, we add "dummy" symbols with probability 0, to fill the tree
- At each reduction stage, the number of symbols reduces by $D - 1$
- $1 + k(D - 1)$ total symbols, with k merges, so add enough dummies
- Avg. length = 1.7 trits = 2.69 bits

x	$p(x)$	$l^*(x) = -\log_3 p(x)$
1	0.25	1.26
2	0.25	1.26
3	0.2	1.47
4	0.1	2.1
5	0.1	2.1
6	0.1	2.1
7	0	—

x	$p(x)$
1	0.25
2	0.25
3	0.2
4	0.1
{5,6,7}	0.2

x	$p(x)$
1	0.25
2	0.25
{3,{5,6,7},4}	0.5

x	$p(x)$
{{3,{5,6,7},4},1,2}	1

$$H(X) = 2.46 \text{ bits}$$

x	$c(x)$	$l(x)$	$p(x)$	$l^*(x)$
1	1	1	0.25	1.26
2	2	1	0.25	1.26
3	00	2	0.2	1.47
4	02	2	0.1	2.1
5	010	3	0.1	2.1
6	011	3	0.1	2.1

Equivalence of Source Coding and “20 Questions”

- Most efficient series of “yes–no” questions to determine an object from a class of objects
- We know the objects PMF, can we find the most efficient sequence of *Q* questions?
  - To determine an object, we need to ensure that the responses to the *Q* questions uniquely identify it from the set of possible objects
    - ❖ it is not necessary that the last question has a “yes” answer
- Any question depends only on the answers to the questions before it
- The sequence of answers uniquely determines the object, each object has a different sequence





Equivalence of Source Coding and "20 Questions"

- Assign "yes" = 1 and "no" = 0 to obtain a binary code for the set of objects
- The average length of this code is the average number of questions E{*Q*}
- Also, from a binary code for the set, we can find a sequence of questions, with the average number of questions equal to the expected CW length
- The first question in this scheme becomes
  - Is the first bit equal to 1 in the object's codeword?
- Since the Huffman code is the best source code for a RV, the optimal series of questions is determined by the Huffman code

Equivalence of Source Coding and "20 Questions"

x	$c(x)$	$l(x)$	$p(x)$	$l^*(x)$
1	01	2	0.25	2
2	10	2	0.25	2
3	11	2	0.2	2.32
4	000	3	0.15	2.74
5	001	3	0.15	2.74

- For the previous binary example
  - The optimal 1st question is "Is $X = 2$ or $X = 3$?"
  - The answer determines the 1st bit of the code
  - If the answer is "yes," the next question should be "Is $X = 3$?"
  - If the answer is "no", the next question should be "Is $X = 1$?": this determines the 2nd bit
  - We can ask our 2nd question "Is $X = 1$ or $X = 3$?" even before knowing the answer to the 1st: the 2nd bit is independent of the 1st
  - Potential match against ChatGPT
    1. Q1: "Is $X = 2$ or $X = 3$?"
    2. Q2: "Is $X = 1$ or $X = 3$?" (fast typist)
    3. A1: yes/no
    4. A2: yes/no
    5. Q3: Could be "Is $X = 4$?" or "Is $X = 5$?"

$$H(X) \leq \mathrm{E}\{Q\} < H(X) + 1$$

Equivalence of Source Coding and "20 Questions"

x	$c(x)$	$l(x)$	$p(x)$	$l^*(x)$
1	01	2	0.25	2
2	10	2	0.25	2
3	11	2	0.2	2.32
4	000	3	0.15	2.74
5	001	3	0.15	2.74

- The 1st bit is
  - **1** for {2,3}
  - **0** for {1,4,5}
- Q1: "Is $X = 2$ or $X = 3$?"
- The 2nd bit is
  - **1** for {1,3}
  - **0** for {2,4,5}
- Q2: "Is $X = 1$ or $X = 3$?" (in parallel with Q1)
  - Q2 is independent of Q1, because the 2nd bit is present in each CW
- The 3rd bit is
  - **1** for {5}
  - **0** for {4}
- Conditional Q3: "Is $X = 5$?"

Huffman Coding for Weighted Codewords

- Huffman's algorithm for minimizing $\sum_i p_i l_i$ can be applied to any set of numbers $p_i \geq 0$, regardless of $\sum_i p_i$
- In this case, the Huffman code minimizes the sum of weighted code lengths $\sum_i w_i l_i$ rather than the average code length
  - Example: minimum weighted sum is 36

x	$w(x)$
1	5
2	5
3	4
4	4

x	$w(x)$
1	5
2	5
{3,4}	8

x	$w(x)$
{1,2}	10
{3,4}	8

x	$w(x)$
{{1,2},{3,4}}	18

$$\sum_i w_i l_i = 2 \cdot 5 \cdot 2 + 2 \cdot 4 \cdot 2 = 36$$

x	$c(x)$
{{1,2},{3,4}}	—

x	$c(x)$
{1,2}	0
{3,4}	1

x	$c(x)$
1	00
2	01
3	10
4	11

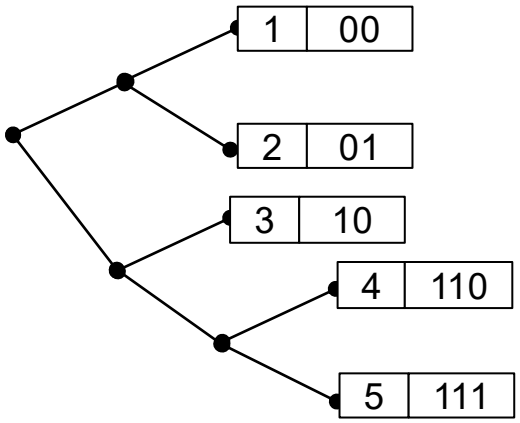
x	$c(x)$	$l(x)$	$w(x)$
1	00	2	5
2	01	2	5
3	10	2	4
4	11	2	4

Huffman and "Slice" Questions (Alphabetic Codes)

- In the 20 questions game, Huffman codes ask arbitrary questions of the form "Is $X \in \mathcal{A}$?" $\forall$ subset $\mathcal{A} \subseteq \mathcal{X} = \{1,2,\ldots,m\}$
- Consider the same game with a restricted set of questions
- Specifically, we assume that the elements of $\mathcal{X} = \{1,2,\ldots,m\}$ are ordered so that $p_1 \geq p_2 \geq \cdots \geq p_m$ and that the only questions allowed are of the form "Is $X > a$?" for some a
  - The code constructed by the Huffman algorithm may not correspond to slices, i.e., sets like $\{x: x < a\}$
- If we take the CW lengths ($l_1 \leq l_2 \leq \cdots \leq l_m$) from the Huffman code and use them to assign the symbols to the code tree by taking the first available node at the corresponding level, we will construct another optimal code
- However, this is a **slice code**, since each question (each code bit) splits the tree into sets of the form $\{x: x > a\}$ and $\{x: x < a\}$

Example

x	$c(x)$	$l(x)$	$p(x)$	$l^*(x)$
1	01	2	0.25	2
2	10	2	0.25	2
3	11	2	0.2	2.32
4	000	3	0.15	2.74
5	001	3	0.15	2.74



- The code that was constructed by the previous Huffman coding procedure is not a slice code
- But using the ordered CW lengths from the Huffman procedure, {2,2,2,3,3}, and assigning the symbols to the first available node on the tree, we obtain the following code for this RV

$$1 \rightarrow 00, \quad 2 \rightarrow 01, \quad 3 \rightarrow 10, \quad 4 \rightarrow 110, \quad 5 \rightarrow 111$$

- This code is a slice code, known as **alphabetic code**, because the codewords are ordered "alphabetically"

Huffman Codes and Shannon codes

- Using CW lengths of $\lceil \log(1/p_i) \rceil$ (**Shannon coding**) may be much worse than the optimal code for some symbol
- E.g., consider 2 symbols with probabilities 0.9999 and 0.0001
  - Using CW lengths of $\lceil \log(1/p_i) \rceil$ gives 1 bit and 14 bits
- The optimal CW length is obviously 1 bit for both symbols
- Hence, the CW for the infrequent symbol is much longer in the Shannon code than in the optimal code
- Is it true that the CW lengths for an optimal code are always less than $\lceil \log(1/p_i) \rceil$? Not always
- Although either the Shannon or the Huffman code can be shorter for individual symbols, the Huffman code is shorter on average
- Also, the Shannon and the Huffman differ by less than 1 bit in expected codelength (since both lie between H and $H + 1$)

Example

- Consider a RV *X* with this PMF
- Huffman coding gives lengths of (1,2,3,3) or (2,2,2,2), depending on the merges
- Both codes achieve the same expected length $L = 2$ ($H = 1.86$)

x	$p(x)$	$l^*(x)$	$\lceil l^*(x) \rceil$
1	1/3	1.59	2
2	1/3	1.59	2
3	1/4	2	2
4	1/12	3.59	4

x	$p(x)$
1	1/3
2	1/3
3	1/4
4	1/12

x	$p(x)$
1	1/3
2	1/3
{3,4}	1/3

x	$p(x)$
{2,{3,4}}	2/3
1	1/3

x	$p(x)$
{{2,{3,4}},1}	1

x	$c(x)$	$l(x)$
1	1	2
2	00	1
3	010	3
4	011	3

- In the 1st code, the 3rd symbol has length 3, which is greater than $\lceil \log(1/p_3) \rceil$

x	$p(x)$
{1,2}	2/3
{3,4}	1/3

x	$p(x)$
{{1,2},{3,4}}	1

x	$c(x)$	$l(x)$
1	00	2
2	01	2
3	10	2
4	11	2

Example

- Shannon coding idea was given in the 1948 paper
  - Order symbol probabilities in decreasing order
  - Accumulate probabilities up to the previous symbol
  - Expand the cumulative value as binary fraction with the required no. of bits

x	$p(x)$	$l^*(x)$	$l(x)$	$\sum p(x)$	$(\sum p(x))_2$	$c(x)$
1	1/3	1.59	2	0.000000000	0.**00**0000…	00
2	1/3	1.59	2	0.333333333	0.**01**0101…	01
3	1/4	2	2	0.666666666	0.**10**1010…	10
4	1/12	3.59	4	0.916666666	0.**1110**10…	1110

- Binary fractions are calculated like this, e.g., for 0.32895
  - 0.32895x2 = 0.6579 → 0
  - 0.6579x2 = 1.3158 → 1
  - 0.3158x2 = 0.6316 → 0
  - 0.6316x2 = 1.2632 → 1

… and so on

Example

- Thus, the CW length for a Shannon could be shorter than the CW length of the corresponding symbol of a Huffman (optimal) code
- In this example, Shannon coding achieves L = 2.167 bits
  - Larger than the Huffman code length
  - Still within 1 bit of the optimal code length
- This example also illustrates the fact that the set of CW lengths for an optimal Huffman code is not unique (there may be more than one set of lengths with the same expected value)

Fano Codes

- Fano proposed a suboptimal procedure for constructing a source code, which is like slice codes
- In his method we first order the probabilities in decreasing order
- Then we choose *k* such that
  $|\sum_{i=1}^{k} p_i - \sum_{i=k+1}^{m} p_i|$ is minimized
- This divides the source symbols into two sets of almost equal probability

x		1	2	3	4	5	$\lvert F_1(x) - F_2(x) \rvert$
$p(x)$		0.25	0.25	0.2	0.15	0.15	
$F_1(x) = \sum_{a \le x} p(a)$	$F_2(x) = \sum_{a > x} p(a)$	0.25	0.75				0.5
		0.5		0.5			0.0
		0.7			0.3		0.4
		0.85				0.15	0.7

Fano Codes

- Assign 0 for the 1st bit of the upper set and 1 to the lower set
- Repeat this process for each subset
- Recursively, we obtain a code for each source symbol
- This scheme, although not optimal in general, achieves $L(C) \leq H(X) + 2$
  - In this example, it has the same length of the Huffman code

x		1	2	3	4	5	$\lvert F_1(x) - F_2(x) \rvert$
$p(x)$		0.25	0.25	0.2	0.15	0.15	
$F_1(x)$	$F_2(x)$	0		1			
		0.25	0.25	0.2	0.3		0.0 | 0.1
				0.35		0.15	0.2

x		1	2	3	4	5	$\lvert F_1(x) - F_2(x) \rvert$
$p(x)$		0.25	0.25	0.2	0.15	0.15	
$F_1(x)$	$F_2(x)$	0		1			
		0	1	0	1		
					0.15	0.15	0.0

x		1	2	3	4	5	$\lvert F_1(x) - F_2(x) \rvert$
$p(x)$		0.25	0.25	0.2	0.15	0.15	
$F_1(x)$	$F_2(x)$	0		1			
		0	1	0	1		
					0	1	
$C(x)$		00	01	10	110	111	

Optimality of Huffman Codes

- There are many optimal codes
  - Inverting all the bits or exchanging two CWs of the same length gives an optimal code

- To prove the optimality of Huffman codes, we first prove some properties of a particular optimal code
- Without loss of generality, we will assume that the PMFs are ordered, so that $p_1 \geq p_2 \geq \cdots \geq p_m$

- A code is optimal if $\sum_i p_i l_i$ is minimal

Canonical Codes

- **Lemma:** For any PMF, there exists an optimal instantaneous **canonical** code (with minimum expected length) where
  1. CW lengths are ordered inversely with the probs.: if $p_j > p_k$, then $l_j \leq l_k$
  2. the two longest CWs have the same length
  3. two of the longest CWs differ only in the last bit and correspond to the two least likely symbols

  If $p_1 \geq p_2 \geq \cdots \geq \mathrm{p}_m$, the optimal code C has $l_1 \leq l_2 \leq \cdots \leq l_{m-1} = l_m$ and $c(x_{m-1})$ and $c(x_m)$ differ only in the last bit

  - *Proof*: The proof amounts to swapping, trimming, and rearranging. Consider an optimal code C_m

  1. If $p_j > p_k$, then $l_j \leq l_k$ → Swap CWs j and k of C_m to obtain C_m'

$$L(C_m') - L(C_m) = \sum_i p_i l_i' - \sum_i p_i l_i = (p_j l_k + p_k l_j) - (p_j l_j + p_k l_k)$$
$$= (p_j - p_k)(l_k - l_j) \geq 0$$

($\geq$ for optimality of C_m; $p_j > p_k$ by definition)

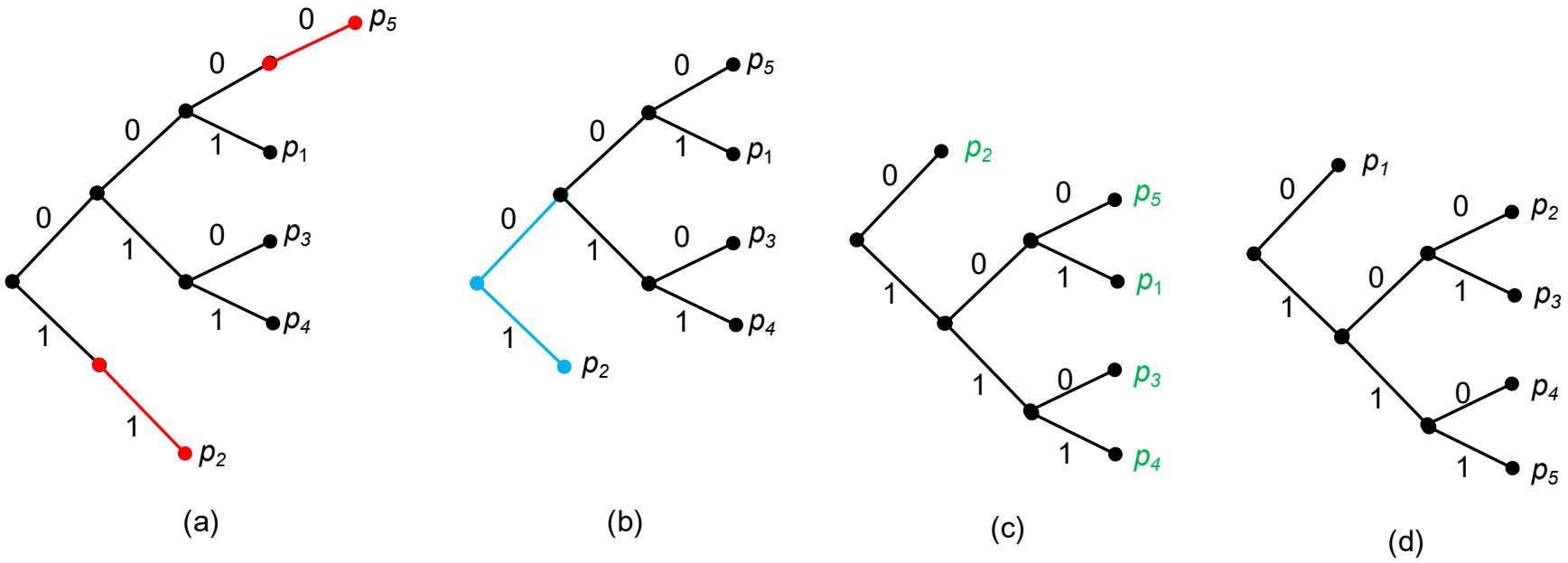
  - ❖ Then $l_k - l_j \geq 0$

Canonical Codes (proof cont'd)

2. The two longest CWs are of the same length → Here we trim the CWs
   - If the two longest CWs have different lengths, delete the last bit of the longest, preserve the prefix property and achieve lower expected CW length
   - Hence, the two longest codewords must have the same length
   - By property 1, the longest CWs must belong to the least probable source symbols

3. The two longest CWs differ only in the last bit and correspond to the two least likely symbols → Not all optimal codes satisfy this property, but by rearranging, we can find an optimal code that does
   - If there is a maximal-length CW without a sibling, we can delete the last bit of the CW and still satisfy the prefix property
   - This reduces the average CW length and contradicts the optimality of the code
   - Hence, every maximal-length CW in any optimal code has a sibling
   - Exchange the longest CWs so that the two least probable source symbols are associated with two siblings on the tree: no change of expected length $\Sigma_i p_i l_i$
   - Thus, the codewords for the two lowest-probability source symbols have maximal length and agree in all but the last bit

Proof cont'd

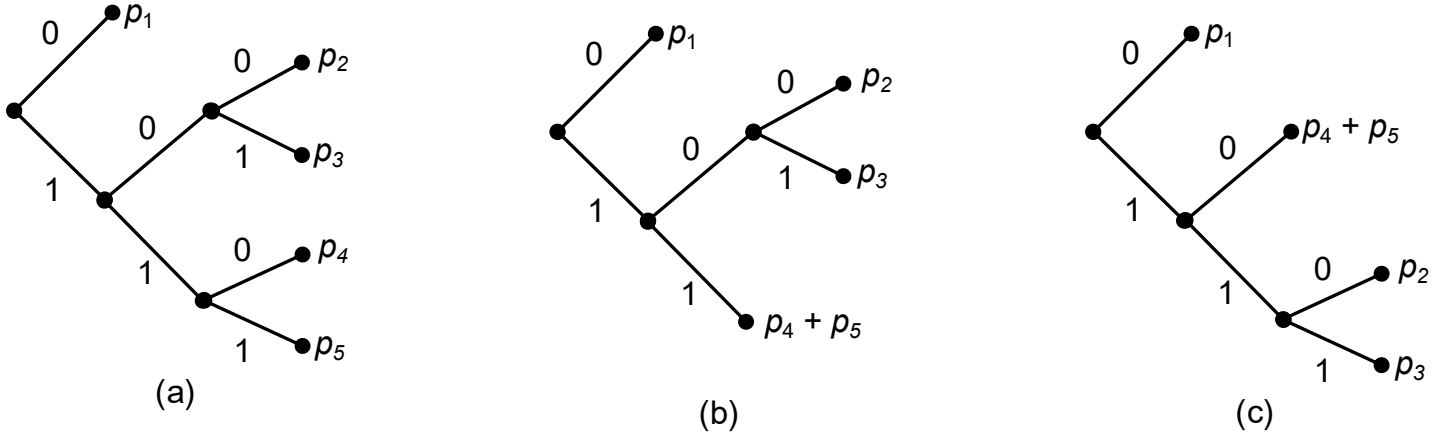


- a) possible instantaneous code
- b) improve code by trimming branches without siblings
- c) rearrange so that lengths are ordered from top (short) to bottom (long)
- d) swap probabilities assignments to improve the expected tree depth

- Every optimal code can be rearranged and swapped into **canonical form** as in (d), where $l_1 \leq l_2 \leq \cdots \leq l_m$, $l_{m-1} = l_m$, and the last two codewords differ only in the last bit

Canonical Codes and Reductions

- For any PMF for an alphabet of size m, $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the **Huffman reduction** $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-2}, p_{m-1} + p_m)$ over an alphabet of size $m-1$



  - a) Canonical optimal code
  - b) combining the two lowest probabilities
  - c) canonical code for m-1 symbols (put probabilities in decreasing order)

Makes the locally optimal choice at each stage

- Huffman is a "greedy" algorithm: it joins the two least likely symbols at each stage (in general, *greedy* ≠ *optimal*)
  - In this case, local optimality ensures global optimality of the code

Theorem on Optimality

- **Theorem:** Huffman coding is **optimal**; that is, if C^* is a Huffman code and C' is any other uniquely decodable code, $L(C^*) \le L(C')$
  - *Proof:* $C_m^*(\mathbf{p})$ is the canonical optimal code for $\mathbf{p}$ and $C_{m-1}^*(\mathbf{p}')$ an optimal code for $\mathbf{p}'$
  - Expand an optimal code $C^*_{m\text{-}1}(\mathbf{p}')$ to construct a code $C_m(\mathbf{p})$
    - Take the CW in $C^*_{m\text{-}1}$ corresponding to $p_{m-1}+p_m$ and extend it by 1 bit to form two new CWs for symbols m-1 and m. Average length becomes

$$\begin{aligned} L(\mathbf{p}) &= p_1 l_1 + \cdots + p_{m-1}(l_{m-1}+1) + p_m(l_{m-1}+1) \\ &= L^*(\mathbf{p}') + 1 \cdot p_{m-1} + 1 \cdot p_m \end{aligned}$$

CW with prob. ($p_{m-1}+p_m$) is expanded with a 0 and a 1 into two sibling CWs: they increase the average length by 1 bit with these probabilities

  - Reduce an optimal canonical code $C^*_m(\mathbf{p})$ to construct a code $C_{m\text{-}1}(\mathbf{p}')$

$$\begin{aligned} L(\mathbf{p}') &= p_1 l_1 + \cdots + (p_{m-1}+p_m)(l_{m-1}-1) \\ &= L^*(\mathbf{p}) - 1 \cdot p_{m-1} - 1 \cdot p_m \end{aligned}$$

Merge the CWs for the two least probable symbols $m-1$ and m with probs. p_{m-1} and p_m

Theorem on Optimality (proof cont'd)

- Sum these 2 equations: extension of optimal code for **p'** is optimal for **p**

$$L(\mathbf{p}) + L(\mathbf{p}') = L^*(\mathbf{p}') + p_{m-1} + p_m + L^*(\mathbf{p}) - p_{m-1} - p_m = L^*(\mathbf{p}') + L^*(\mathbf{p})$$

≥0 for optimal *L** ≥0 for optimal *L**

$$\big(L(\mathbf{p}') - L^*(\mathbf{p}')\big) + \big(L(\mathbf{p}) - L^*(\mathbf{p})\big) = 0 \quad \Longrightarrow \quad L(\mathbf{p}) = L^*(\mathbf{p})$$

So, this difference must be 0 And this one too

- Start with an (obviously optimal) two elements code and go on by induction with the code extensions

$$\mathbf{p}' = (p_1, p_2 + p_3) \rightarrow \mathbf{p} = (p_1, p_2, p_3)$$

$$\mathbf{p}' = (p_1, p_2, p_3 + p_4) \rightarrow \mathbf{p} = (p_1, p_2, p_3, p_4)$$

$$\dots$$

$$\mathbf{p}' = (p_1, \dots, p_{m-1} + p_m) \rightarrow \mathbf{p} = (p_1, \dots, p_{m-1}, p_m)$$

Shannon-Fano-Elias Code

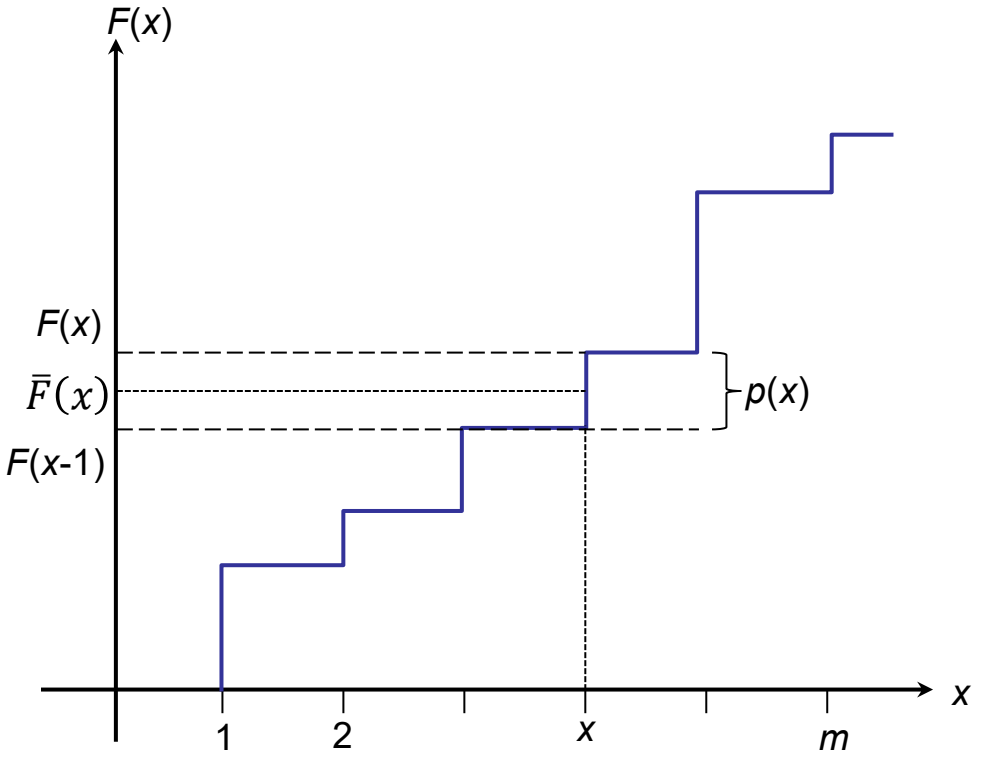
- CW lengths $l(x) = \lceil \log[1/p(x)] \rceil$ satisfy the Kraft inequality and can therefore be used to construct a uniquely decodable code
  - A simple procedure uses the cumulative distribution function (CDF)
- Let $\mathcal{X}=\{1,2,\ldots,m\}$ and $p(x)>0\ \forall x$, with CDF

$$F(x) = \sum_{a \le x} p(a)$$

and modified CDF

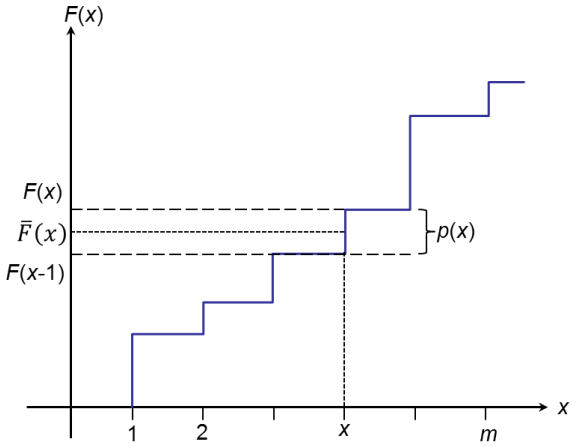
$$\bar{F}(x) = \sum_{a<x} p(a) + \frac{1}{2} p(x)$$

  - In the last term of the sum, add half probability only
  - Midpoint between 2 steps along the vertical axis of the CDF



Shannon-Fano-Elias Code

- Since $p(x)>0$, $\bar{F}(a) \neq \bar{F}(b)$ if $a \neq b$, hence we have x if we know $\bar{F}(x)$
  - Thus, $\bar{F}(x)$ can be used as a code for x
- But $\bar{F}(x)$ is real, needs infinite bits
  - Approximate with how much accuracy?



- Truncate $\bar{F}(x)$ to $l(x)$ bits, $\lfloor \bar{F}(x) \rfloor_{l(x)}$, and use them as a code for x, so that it lies within the step corresponding to x
  - $l(x) = \left\lceil \log \frac{1}{p(x)} \right\rceil + 1$ bits suffice to describe x
- Truncation keeps values distinct

$$\bar{F}(x) - \lfloor \bar{F}(x) \rfloor_{l(x)} < \frac{1}{2^{l(x)}} = 2^{-l(x)} \leq \frac{p(x)}{2} = \bar{F}(x) - F(x-1)$$

-⌈a⌉=⌊-a⌋

$$2^{-l(x)} = 2^{-\left\lceil \log \frac{1}{p(x)} \right\rceil - 1} = \frac{2^{-\left\lceil \log \frac{1}{p(x)} \right\rceil}}{2} = \frac{2^{\left\lfloor -\log \frac{1}{p(x)} \right\rfloor}}{2} = \frac{2^{\lfloor \log p(x) \rfloor}}{2} \leq \frac{2^{\log p(x)}}{2} = \frac{p(x)}{2}$$

Shannon-Fano-Elias Code

- To check that the code is prefix-free, consider each CW $z_1 z_2 \cdots z_l$ to represent the interval $[0.z_1 z_2 \cdots z_l, 0.z_1 z_2 \cdots z_l + 1/2^l)$
  - It is prefix-free iff CW intervals are disjoint
- Interval length $2^{-l(x)}$ is less than ½ CDF jump ($F(x) - F(x-1)$) of x
  - The lower end of the interval is in the lower half of the step
  - The upper end lies below the top of the step
  - Any interval lies entirely within the step of that symbol in the CDF
- Therefore, the intervals corresponding to different codewords are disjoint and the code is prefix-free
  - This does not require the symbols to be ordered by decreasing probability
- Average CW length stays within 2 bits of the entropy

$$L = \sum_x p(x) l(x) = \sum_x p(x) \left(\left\lceil \log \frac{1}{p(x)} \right\rceil + 1 \right) < H(X) + 2$$

Example: Dyadic Case (2-adic)

x	$p(x)$	$F(x)$	$\bar{F}(x)$	$\bar{F}(x)$ in binary	$l(x) = \left\lceil \log \frac{1}{p(x)} \right\rceil + 1$	Codeword
1	0.25	0.25	0.125	0.001	3	001
2	0.5	0.75	0.5	0.10	2	10
3	0.125	0.875	0.8125	0.1101	4	1101
4	0.125	1.0	0.9375	0.1111	4	1111

0.1250x2=0.2500 → 0
0.2500x2=0.5000 → 0
0.5000x2=1.0000 → 1

0.5000x2=1.0000 → 1
0.0000x2=0.0000 → 0

0.8125x2=1.6250 → 1
0.6250x2=1.2500 → 1
0.2500x2=0.5000 → 0
0.5000x2=1.0000 → 1

0.9375x2=1.8750 → 1
0.8750x2=1.7500 → 1
0.7500x2=1.5000 → 1
0.5000x2=1.0000 → 1

- The average CW length is L=2.75 bits (entropy is H=1.75 bits)
  - The Huffman code achieves the entropy bound L=1.75
- Looking at the CWs, it is obvious that there is some inefficiency
  - E.g., the last bit of the last two CWs can be omitted
  - If we remove the last bit from <u>all</u> the CWs, the code is no longer prefix-free

1 001
2 10
3 1101
4 1111
$1.75 \le L(X) = 2.75 \not< 1.75+1$
$\Sigma_i 2^{-l_i} = 0.75 \le 1$

1 001
2 10
3 110
4 111
$1.75 \le L(X) = 2.5 < 1.75+1$
$\Sigma_i 2^{-l_i} = 0.625 \le 1$

1 00
2 1
3 110
4 111
$1.75 \le L(X) = 1.75 < 1.75+1$
$\Sigma_i 2^{-l_i} = 1 \le 1$

Example: Nondyadic Case

x	$p(x)$	$F(x)$	$\bar{F}(x)$	$\bar{F}(x)$ in binary	$l(x) = \left\lceil \log \frac{1}{p(x)} \right\rceil + 1$	Codeword
1	0.25	0.25	0.125	0.001	3	001
2	0.25	0.5	0.375	0.011	3	011
3	0.2	0.7	0.6	0.1001…	4	1001
4	0.15	0.85	0.775	0.1100…	4	1100
5	0.15	1.0	0.925	0.1110…	4	1110

0.125x2=0.250 → 0
0.250x2=0.500 → 0
0.500x2=1.000 → 1

0.375x2=0.750 → 0
0.750x2=1.500 → 1
0.500x2=1.000 → 1

0.600x2=1.200 → 1
0.200x2=0.400 → 0
0.400x2=0.800 → 0
0.800x2=1.600 → 1
…

0.775x2=1.550 → 1
0.550x2=1.100 → 1
0.100x2=0.200 → 0
0.200x2=0.400 → 0
…

0.925x2=1.850 → 1
0.850x2=1.700 → 1
0.700x2=1.400 → 1
0.400x2=0.800 → 0
…

- Since the distribution is not dyadic, the representation of $\bar{F}(x)$ in binary may have an infinite number of bits
- The above code is 1.2 bits longer (L=3.5), on the average, than the Huffman code for this source (L=2.3), which has H=2.286

Competitive Optimality as a Game

- Shannon (Huffman) codes are optimal under many criteria
- One example is **competitive optimality**, a zero-sum game
- Two players are given a PMF and are asked to design an instantaneous code
- Then a source symbol is drawn from this PMF
- Payoff
  - +1 if CW is shorter than that of the other player, $l(x) < l'(x)$
  - 0 if the two players propose equal-length CWs, $l(x) = l'(x)$
  - -1 if CW is longer than that of the other player, $l(x) > l'(x)$



Shannon (Huffman) is Competitively Optimal

- For Shannon (Huffman) codes with average length $l(X)$

$$\Pr\{l(X) \geq l'(X) + c\} \leq \frac{1}{2^{c-1}}$$

- Hence, no competing code with length $l'(X)$ can do much better than the Shannon (Huffman) code most of the time
- Example
  - The competitor wins by c=5 bits in less than $2^{-(5-1)}$=1/16 of the cases
- Moreover

$$\Pr\{l(X) < l'(X)\} \geq \Pr\{l(X) > l'(X)\}$$

- Strong robustness with respect to the payoff, not just average optimality
- An extension of Shannon codes, arithmetic codes, are nowadays used in many compression systems